

QVerse — Week 1: Classical vs Quantum Computing — Your First Qiskit Circuits

Theory Booklet | Introduction to Quantum Computing & Programming

QVerse

2026

How this booklet fits the course

This theory booklet is designed to be read alongside `01_orientation_first_qiskit_circuits.ipynb`. It contains the conceptual and mathematical material that should be explained during Week 1; the notebook then turns those ideas into experiments, circuit construction, and exercises.

Learning objectives

- Distinguish bits from qubits at an operational level.
- Recognize circuit, qubit, gate, measurement, shot, simulator, and QPU.
- Create, draw, and sample a one-qubit Qiskit circuit.
- Use the X gate and computational-basis measurement.

Recommended workflow

1. Read the relevant theory section.
2. Predict the result of the corresponding notebook experiment.
3. Run the notebook.
4. Explain any difference between the prediction and the output.
5. Complete the core exercises without treating the solution notebook as a substitute for reasoning.

1. Why this week matters

Quantum programming becomes much easier once the circuit model is clear. The objective of the first week is not to learn advanced quantum mechanics. It is to learn the vocabulary used throughout the course and to understand what a quantum circuit *does* operationally.

A classical program manipulates classical data such as bits, integers, and floating-point numbers. A gate-based quantum program manipulates **qubits** using **quantum gates**, and then extracts classical information through **measurement**. The quantum part of the computation happens before the final measurement.

The central workflow is:

1. **Prepare** an initial state.
2. **Transform** it with quantum gates.
3. **Measure** selected qubits.
4. **Repeat** the experiment many times when the outcome is probabilistic.
5. **Interpret** the observed distribution.

This same prepare-transform-measure pattern will reappear in every algorithm in the course.

2. Classical bits and qubits

A classical bit has one of two values:

0 or 1.

A qubit has two distinguished computational-basis states, written

$$|0\rangle, \quad |1\rangle.$$

The ket notation $|\cdot\rangle$ is just a compact way of naming quantum states. At this stage, the important fact is that a qubit can be prepared in $|0\rangle$, in $|1\rangle$, or in a more general state that will be introduced in Week 2.

A useful first mental model is:

- a classical bit stores a definite binary value;
- a qubit is a physical two-level quantum system whose state may require amplitudes to describe;
- when measured in the computational basis, the output is still an ordinary classical bit.

Do **not** summarize a qubit by saying “it is both 0 and 1 at the same time.” That phrase hides the role of amplitudes, phase, and measurement. A more accurate statement is that a qubit may occupy a coherent superposition of the computational-basis states.

3. The circuit model

A circuit diagram is read from left to right. Each horizontal wire represents a qubit. Gates are operations applied at specific points in the circuit. A measurement symbol converts the final quantum state into classical data.

The vocabulary used throughout the repository is:

- **Qubit:** the basic quantum information carrier.
- **Quantum register:** a collection of qubits.
- **Gate:** a reversible quantum transformation.
- **Circuit:** an ordered sequence of gates and measurements.
- **Measurement:** conversion of quantum information into a classical outcome.
- **Shot:** one complete execution of a measured circuit.
- **Counts:** the number of times each classical bit string is observed over many shots.
- **Simulator:** classical software that mathematically emulates a quantum circuit.
- **QPU:** a quantum processing unit, meaning real quantum hardware.

A circuit is an abstract logical object. It is not yet tied to a specific physical technology such as superconducting qubits, trapped ions, neutral atoms, or photons.

4. Initialization

In the Qiskit circuit model used in the course, a newly created qubit starts in the state

$$|0\rangle.$$

Therefore, an empty one-qubit circuit followed by measurement should return the classical outcome 0 with probability 1 in an ideal simulator.

This simple case is important because it establishes a baseline. Before adding gates, students should know what the untouched circuit does.

For several qubits, the default initial state is

$$|00 \cdots 0\rangle.$$

Week 5 will formalize the notation for multi-qubit states.

5. The X gate

The Pauli- X gate acts as a quantum NOT operation on the computational-basis states:

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle.$$

Its matrix representation is

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

The matrix formalism will be developed in Week 2. For now, the useful operational rule is that X flips $|0\rangle \leftrightarrow |1\rangle$.

Applying X twice returns the original state:

$$X^2 = I,$$

where I is the identity operation. Therefore:

$$|0\rangle \xrightarrow{X} |1\rangle \xrightarrow{X} |0\rangle.$$

This is the first example of a **gate identity**. Later, gate identities will be useful for simplifying and comparing circuits.

6. Measurement in the computational basis

A computational-basis measurement asks: “Is the qubit observed as 0 or 1?”

For a state that is exactly $|0\rangle$, the result is deterministically 0. For $|1\rangle$, the result is deterministically 1. For superposition states, introduced later, the result is probabilistic.

Measurement is not simply a way to “look at” a quantum state without changing it. In the standard projective model used here, measurement produces a classical result and projects the state into a compatible measurement outcome.

This means that a statevector and a measurement result are different objects:

- the **statevector** describes the quantum state before measurement;
- the **measurement outcome** is one sampled classical result.

7. Why repeated shots exist

If a circuit has a probabilistic result, one run is not enough to estimate the underlying probabilities.

Suppose a circuit ideally produces 0 and 1 with equal probability. Ten shots may produce 6 zeros and 4 ones. One thousand shots may produce 503 zeros and 497 ones. Neither result means the theoretical probability changed.

The shot count controls sampling precision. Week 8 will quantify the statistical error. For now, remember:

$$\text{probability} \neq \text{finite-shot frequency}.$$

The frequency approaches the probability as the number of independent shots becomes large.

8. Simulator versus real hardware

An **ideal statevector simulation** keeps track of the exact mathematical amplitudes of the state. It does not reproduce all the imperfections of a physical processor.

A **shot-based simulator** samples measurement outcomes according to the ideal circuit probabilities.

A **real QPU** additionally introduces physical effects such as:

- gate errors;
- readout errors;
- decoherence;
- restricted connectivity;
- calibration drift.

These effects are postponed until Week 14 so that the logic of the algorithms is learned first.

9. Qiskit concepts used in the notebook

The Week 1 notebook introduces only a small part of Qiskit:

- `QuantumCircuit(n)` creates a circuit with n qubits.
- `x(q)` applies an X gate.
- `measure_all()` adds computational-basis measurements.
- drawing the circuit gives a visual representation.
- a sampler executes the measured circuit repeatedly and returns counts.

The exact software syntax matters less than the conceptual mapping:

quantum idea $\longrightarrow$ circuit $\longrightarrow$ execution $\longrightarrow$ classical data.

Students should always make a prediction before running a circuit. This habit becomes essential once circuits are too complicated to debug by inspection.

10. Reusable circuit-building functions

A useful programming pattern is to wrap circuit preparation inside functions. For example, conceptually a function `make_bit(value)` should:

1. create a one-qubit circuit;
2. leave the qubit unchanged if `value == 0`;
3. apply X if `value == 1`;
4. add measurement;
5. return the circuit.

This pattern separates **what state should be prepared** from **how the experiment is executed**. The same idea will later be used for oracles, ansätze, and algorithm subroutines.

11. Classical NOT versus quantum X

For computational-basis inputs, classical NOT and quantum X have the same truth table:

Input	Output after NOT / X
0	1
1	0

However, X is a linear unitary operation defined on *all* one-qubit states, not only on the two basis states. This distinction becomes important once superpositions are introduced.

Therefore, saying “ X is a NOT gate” is useful but incomplete. It is a quantum operation whose action on $|0\rangle$ and $|1\rangle$ reproduces classical NOT.

12. Common misconceptions to remove immediately

“Quantum computers try every answer and then show all of them”

A quantum state can encode many amplitudes, but measurement does not reveal an exponentially long list of amplitudes. Algorithms must create an interference pattern that makes useful information likely to appear in the final measurement.

“A simulator is a quantum computer”

A simulator is ordinary classical software implementing the mathematics of a quantum system. It is extremely useful for learning and debugging, but its resource requirements grow rapidly with the number of qubits.

“A quantum result should be identical every time”

Only deterministic circuits have identical ideal outcomes. Many quantum experiments are intrinsically probabilistic.

“More shots make the quantum state better”

Shots improve the statistical estimate of measurement probabilities. They do not improve the state preparation itself.

13. What to know before opening the notebook

You should be able to explain, without code:

- the difference between a bit and a qubit;
- what a gate is;
- why a circuit has to be measured to obtain classical output;
- what a shot is;
- what X does to $|0\rangle$ and $|1\rangle$;
- why $X^2 = I$;
- the difference between an ideal simulator and a physical QPU.

The notebook then turns these ideas into executable circuits.

Suggested references

- IBM Quantum Learning, *Basics of Quantum Information* and current Qiskit learning material: <https://quantum.cloud.ibm.com/learning/en/courses>
- IBM Quantum Documentation and Qiskit guides: <https://quantum.cloud.ibm.com/docs>
- Quantum Rings, *Quantum 101*: <https://www.quantumrings.com/course/quantum101>
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press.

The software-facing details in the practical notebook should be treated as the canonical implementation for this course. The purpose of this booklet is to explain the physics, mathematics, and algorithmic reasoning behind that implementation.